CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Patrones arquitectónicos clásicos

Diseño y Patrones Arquitectónicos · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 2

Patrones arquitectónicos clásicos

Cinco patrones sostienen casi todo

¿Qué patrón usa el sistema con el que usted trabaja hoy? Probablemente más de uno de los cinco de esta unidad.

- **Surgieron entre 1970 y 1990**
- **Siguen dentro de los sistemas modernos**
- **Un microservicio por dentro suele ser clásico**

Lo que podrá hacer al terminar

- Explicar los cinco patrones clásicos
- Comparar ventajas y costos de cada uno
- Distinguir MVC de la arquitectura en capas
- Elegir el patrón de la versión 1
- Preparar el Hito 2 del proyecto

01

1. Qué tienen en común los clásicos

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Soluciones probadas, vocabulario común

Un patrón clásico define la organización fundamental del sistema: cómo interactúan los componentes, cómo se reparten y cómo fluyen los datos.

Reducir incertidumbre

esquemas predecibles para diseñar

Mantenibilidad

separar preocupaciones

Escalabilidad

dividir responsabilidades

Portabilidad

abstraer detalles en capas

Los cinco de un vistazo

Patrón	Organiza	Comunicación	Favorece
Capas	Niveles apilados	Hacia la capa de abajo	Mantenibilidad
Cliente-servidor	Quien pide y quien provee	Solicitud y respuesta	Centralización
MVC	Modelo, vista y controlador	Eventos del usuario	Varias vistas
Pipes and filters	Etapas de procesamiento	Tuberías en un sentido	Reutilizar etapas
Peer-to-peer	Nodos iguales	Directa entre pares	Resiliencia

02

2. Arquitectura por capas

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Cada capa, una responsabilidad

Presentación · muestra datos y captura acciones

Negocio · reglas y casos de uso

Persistencia · acceso a datos detrás de interfaces

Datos · base de datos y almacenamiento

Las dependencias bajan, nunca suben

En capas estrictas cada capa habla solo con la inmediata inferior. En capas relajadas puede saltar niveles, pero con reglas escritas.

- **Superior depende de inferior**
- **La inferior no conoce la superior**
- **Sin ciclos de dependencia**
- **Saltos solo con regla explícita**

Capas en código

JAVA

```
class PedidoController {   // presentación: solo traduce
  Pedido crear(PedidoDto d) { return servicio.registrar(d); } }
class PedidoService {      // negocio: precios y cierre del jueves
  Pedido registrar(PedidoDto d) { validarCierre(d); return repo.guardar(d); } }
interface PedidoRepository { Pedido guardar(PedidoDto d); }   // datos
```

Ventajas y costos de las capas

Ventajas

- Cambios localizados
- Escalar una capa
- Reutilizar capas
- Equipos en paralelo

Costos

- Latencia al cruzar capas
- Exceso de capas y reglas
- Túneles entre capas lejanas

03

3. Cliente-servidor

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Uno pide, otro provee

Cliente
inicia la solicitud

Protocolo
HTTP con GET o POST

Servidor
procesa y responde

Recurso
datos centralizados

Escalar el servidor de dos formas

Vertical

- Más CPU y memoria
- Simple de hacer
- Tiene techo físico

Horizontal

- Más servidores
- Balanceador de carga
- Más complejo de operar

Ventajas y costos del cliente-servidor

Ventajas

- Datos centralizados
- Seguridad en un punto
- Lógica sin tocar clientes
- Varios tipos de cliente

Costos

- Punto único de fallo
- Cuellos de botella
- Depende de la red
- Despliegue especializado

PREGUNTA

Si el servidor se cae, ningún cliente tiene servicio. ¿Qué desventaja ilustra?

A Acoplamiento de formatos

B Punto único de fallo

C Túneles entre capas

D Dependencia de la participación

04

4. Modelo-Vista-Controlador

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Tres roles que no se mezclan

Modelo

datos y reglas de negocio

Vista

muestra y captura; es pasiva

Controlador

coordina modelo y vista

El ciclo de una interacción MVC

Usuario
actúa en la vista

Controlador
interpreta el evento

Modelo
actualiza el estado

Vista
se refresca

MVC y capas no son lo mismo

MVC organiza la presentación; las capas organizan todo el sistema. Lo habitual es MVC dentro de la capa de presentación.

- **Capas: sistema completo**
- **MVC: interfaz de usuario**
- **Se combinan, no compiten**

Ventajas y costos de MVC

Ventajas

- Cambiar la vista sin tocar reglas
- Probar el modelo sin interfaz
- Varias vistas, un modelo

Costos

- Excesivo en proyectos pequeños
- Controladores con lógica
- Curva de aprendizaje

PREGUNTA

Un controlador calcula impuestos y descuentos. ¿Qué principio de MVC se rompe?

A Comunicación por protocolos estándar

B Separación de responsabilidades

C Descentralización

D Flujo unidireccional de datos

05

5. Pipes and filters

CORHUILA
CORPORACION UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Datos que fluyen por filtros

Leer
archivo de lecturas

Limpiar
quita registros vacíos

Validar
aplica umbrales

Cargar
guarda en la base

Cuatro principios del pipeline

Etapas atómicas

una responsabilidad por filtro

Un solo sentido

entrada, filtro, salida

Independencia

el filtro no conoce a sus vecinos

Formato común

texto, JSON o bytes

Un pipeline en código

JAVA

```
interface Filtro<T> { List<T> procesar(List<T> entrada); }
var pipeline = List.of(new Limpiar(), new Validar(), new Normalizar());
List<Lectura> datos = leer("lecturas.csv");
for (Filtro<Lectura> f : pipeline) datos = f.procesar(datos);
guardar(datos);
```

Ventajas y costos del pipeline

Ventajas

- Escalar un filtro
- Cambiar un filtro aislado
- Probar cada etapa
- Reordenar etapas

Costos

- Serializar entre etapas
- Formato atado a todos
- Malo con bifurcaciones
- Un filtro lento frena todo

PREGUNTA

¿Por qué cambiar el formato de los datos es costoso en un pipeline de filtros?

A Porque obliga a reiniciar el servidor central

B Porque los filtros comparten estado global

C Porque todos los filtros dependen del formato común de la tubería

D Porque los filtros conocen a sus vecinos

06

6. Peer-to-peer

CORHUILA
CORPORACION UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Todos son cliente y servidor

Cada nodo pide y a la vez provee. No hay una entidad central que controle la red.

- **Descentralización**
- **Autonomía de cada nodo**
- **Cooperación para un objetivo común**
- **Descubrimiento sin intermediarios**

P2P resiste, pero cuesta coordinar

Ventajas

- Sin punto central de fallo
- Descarga de varias fuentes
- Bajo costo de servidores

Costos

- Seguridad difícil de imponer
- Consistencia sin servidor
- Depende de que todos compartan

07

7. Comparar y elegir

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Los cinco frente a frente

Patrón	Favorece	Cuesta	Conviene cuando
Capas	Mantenibilidad	Latencia	Hay reglas de negocio que cambian
Cliente-servidor	Centralización	Punto único de fallo	Varios clientes usan los mismos datos
MVC	Varias vistas	Complejidad inicial	La interfaz cambia más que las reglas
Pipes and filters	Reutilizar etapas	Formato rígido	El problema es transformar datos
Peer-to-peer	Resiliencia	Consistencia	No puede haber un centro

Elegir parte del diagnóstico

1. **Atributo** — el que manda en el reto
2. **Candidatos** — los tres del diagnóstico
3. **Descarte** — el que castiga ese atributo
4. **Elección** — patrón dominante y secundario
5. **Registro** — la decisión y su porqué

Combinar es lo normal

La mayoría de los sistemas web son cliente-servidor por fuera, capas por dentro y MVC en la presentación.

Cliente-servidor · navegador y servidor por HTTP

Capas · presentación, negocio y datos

MVC · dentro de la presentación

PREGUNTA

¿Qué combinación de patrones describe mejor una aplicación web típica?

A Peer-to-peer con pipes and filters

B Cliente-servidor por fuera, capas por dentro y MVC en la presentación

C Solo MVC para todo el sistema

D Pipes and filters para la interfaz de usuario

08

8. Mercado Campesino: la versión 1

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

La v1 de Mercado Campesino

Presentación (MVC) · controladores y vistas web

Negocio · precios, pedidos y cierre del jueves

Persistencia · repositorios

Datos · base de datos relacional

Reglas que el código respeta

- **Controladores sin reglas de negocio**
- **El negocio no conoce la base**
- **Dependencias solo hacia abajo**
- **Una prueba por regla de negocio**

Un caso de uso de punta a punta

Cliente
hace el pedido

Controlador
valida la forma

Servicio
aplica precios y cierre

Repositorio
guarda el pedido

09

9. Hito 2 del proyecto de aula

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Qué entrega la Actividad 2

Pide el enunciado	Con su reto
Selección del problema	El reto del equipo, delimitado
Diseño UML	Componentes, clases y una secuencia
Prototipo (MVP)	La v1 en un repositorio que se ejecuta
Análisis de decisiones	Por qué ese patrón y no los otros
Reflexión crítica	Qué límites ya muestra la v1

Componentes, clases y secuencia

Componentes

piezas y dependencias permitidas

Clases

el modelo del dominio

Secuencia

un caso de uso de punta a punta

Antes de la v2, etiquete la v1

La Actividad 3 compara la v2 contra esta v1. Si la v1 no se puede ejecutar, no hay comparación.

- **git tag v1**
- **git push --tags**
- **README con instrucciones de ejecución**
- **Commits de todos los integrantes**

Errores que cuestan nota

- **Controladores con reglas de negocio**
- **Capas que se saltan sin regla**
- **UML que no coincide con el código**
- **Repositorio sin instrucciones de ejecución**

PREGUNTA

¿Qué hacer antes de empezar la v2?

A Borrar la v1 para no confundir

B Crear un repositorio nuevo

C Etiquetar la v1 en git para poder ejecutarla después

D Copiar la v1 en una carpeta llamada old

Tres ideas para llevarse

Cada patrón favorece algo

y castiga otra cosa

Se combinan

cliente-servidor, capas y MVC

La v1 se etiqueta

es la base de la comparación

Bibliografía de la unidad

- Buschmann et al. (1996)
- Shaw y Garlan (1996)
- Bass, Clements y Kazman (2021)
- Fowler (2002)
- Richards y Ford (2020)

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com